Neural networks

Ens'IA

Ensimag 2021-2022

14 octobre 2021

Ens'IA

Who are we ?

- Association founded in may 2019
- Promote artificial intelligence and its learning
- Share knowledge between students

Ens'IA

Who are we ?

- Association founded in may 2019
- Promote artificial intelligence and its learning
- Share knowledge between students

Les membres :

- Hugo Cartigny (2AA) - President
- Maxime Nemo (ISI) - Vice president
- Clément Dauvilliers (MMIS) - Vice president
- Chaimae Menouar (MMIS) - Treasurer
- Stepan Lebedev (ISI) - Secretary
- Valentin Barbaza (ISI) - Vice Secretary

Ens'IA

Why join us ?

- Showing off at the coffee machine
- Impress your grandparents
- Add a line to your resume
- Eventually learn to do AI

No need to be an expert to help us !

Discord

Join us on Discord !
Useful to ask questions, contact us or to pass on information ! → https ://discord.gg/UgTRbRFqNv

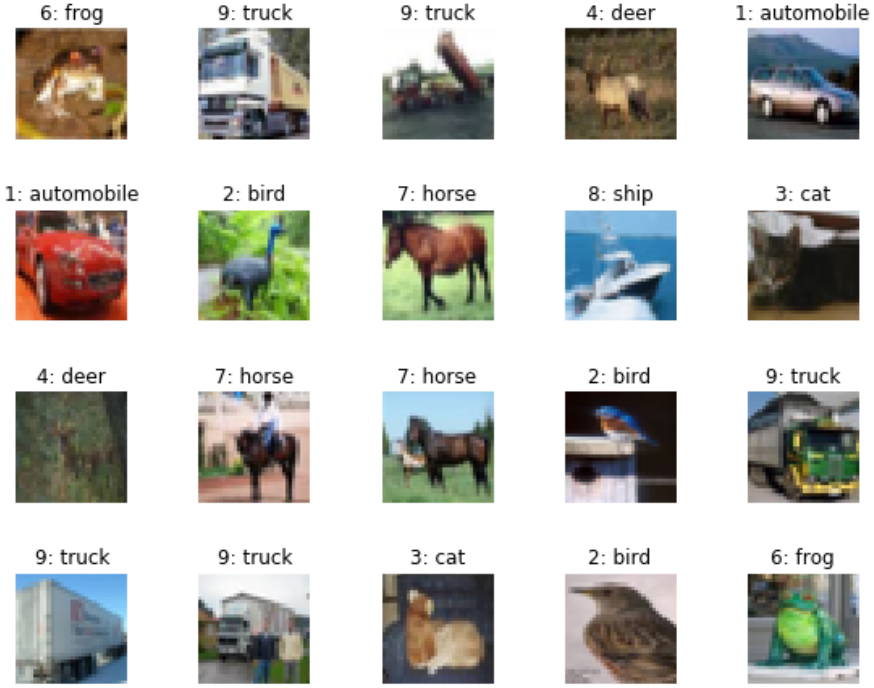
Ens'IA

The program :

- Introduction
- **My first neuron**
- My first neural network
- My first convolution
- AI challenge
- Reinforcement Learning ?
- GANs ?

And after ? → you tell us !

Reminder

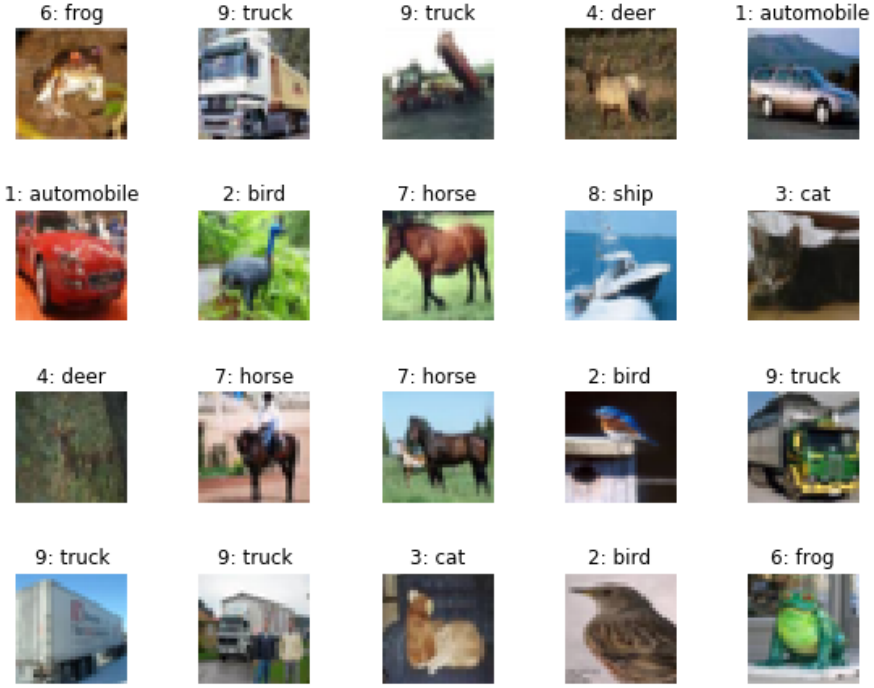


Images classification

→ Supervised learning : x and y known

→ Unsupervised learning : only x

Reminder



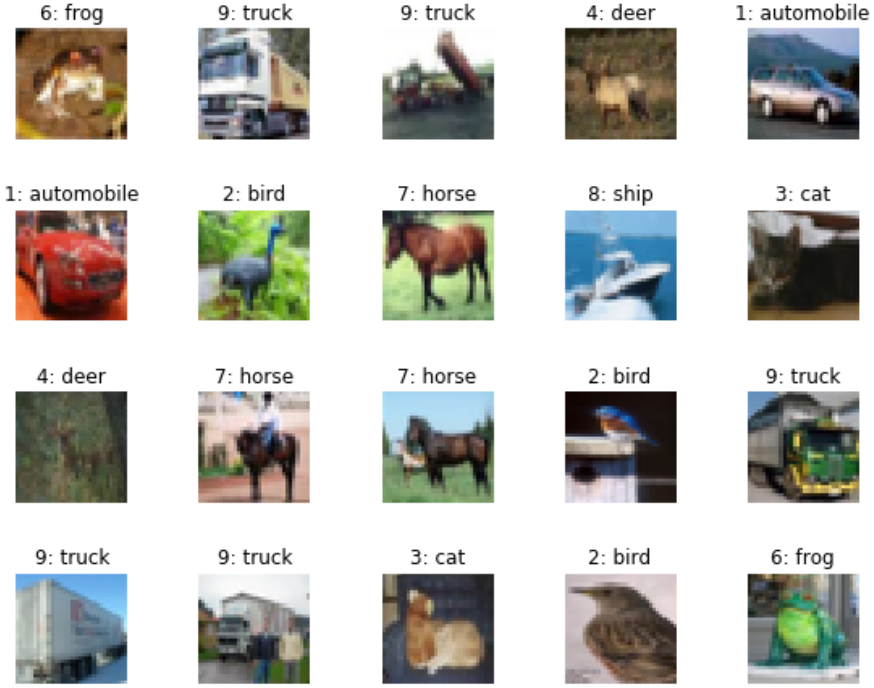
Images classification

→ Supervised learning : x and y known

→ Unsupervised learning : only x

Performances ?

Reminder



Images classification

→ Supervised learning : x and y known

→ Unsupervised learning : only x

Performances ? Not so good ¯_(ツ)_/¯

Goal

$$x \to y$$

Goal

$$f(x) = y$$

Goal

$$f(x) = y$$

How to find f ?

Goal

$$f(x) = y$$

How to find f ?

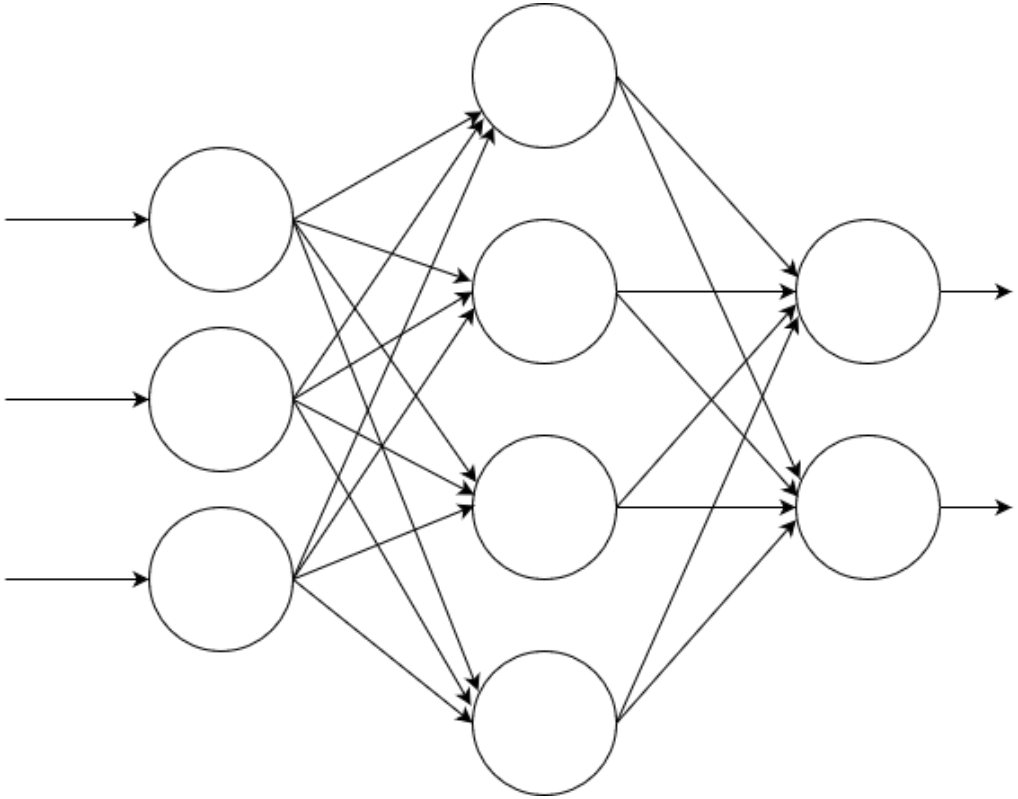
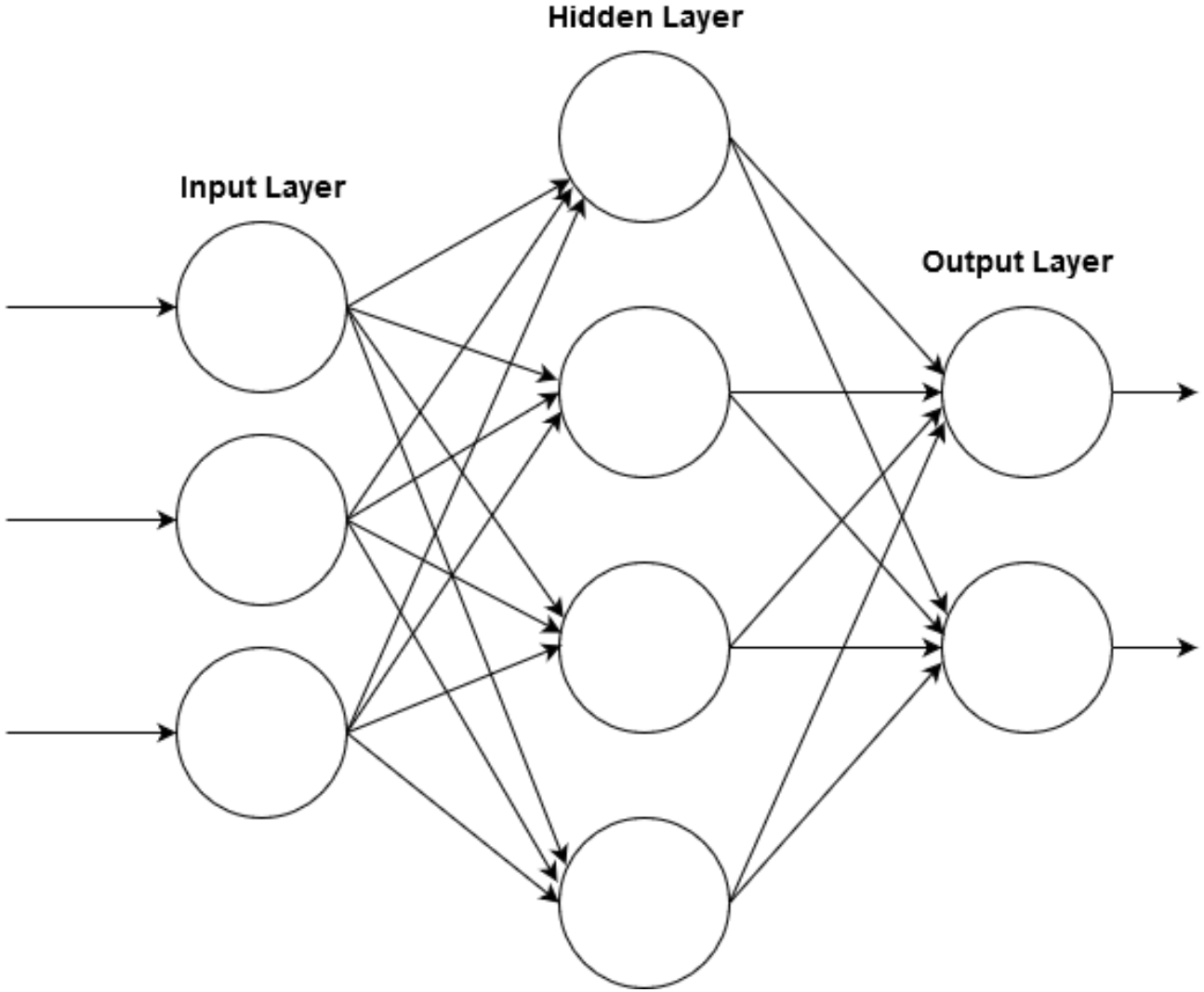


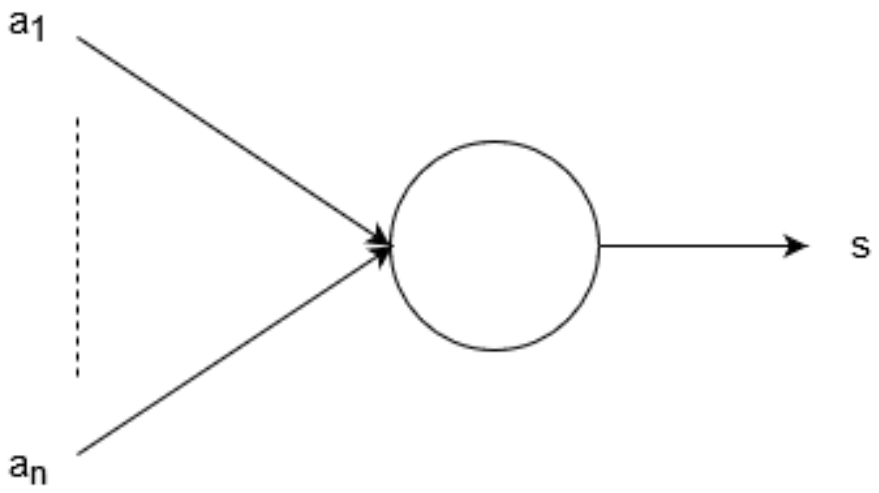
FIGURE 1 – Neural network

Neural network



→ Succession of neuron layers

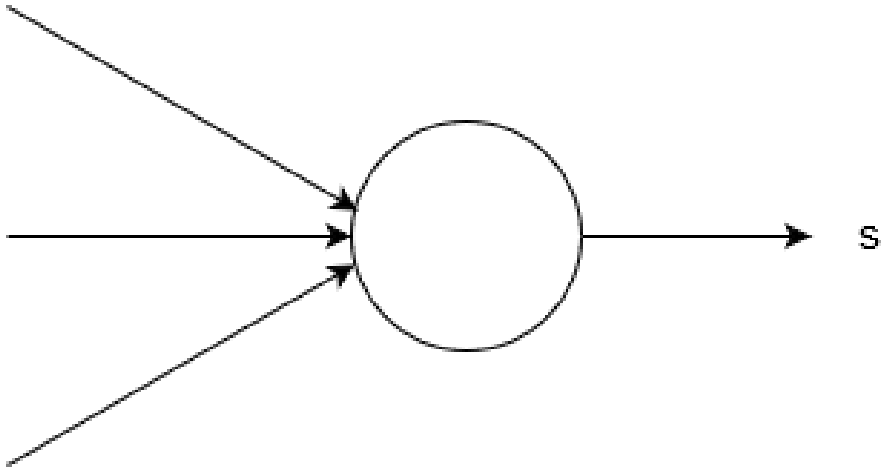
Perceptron



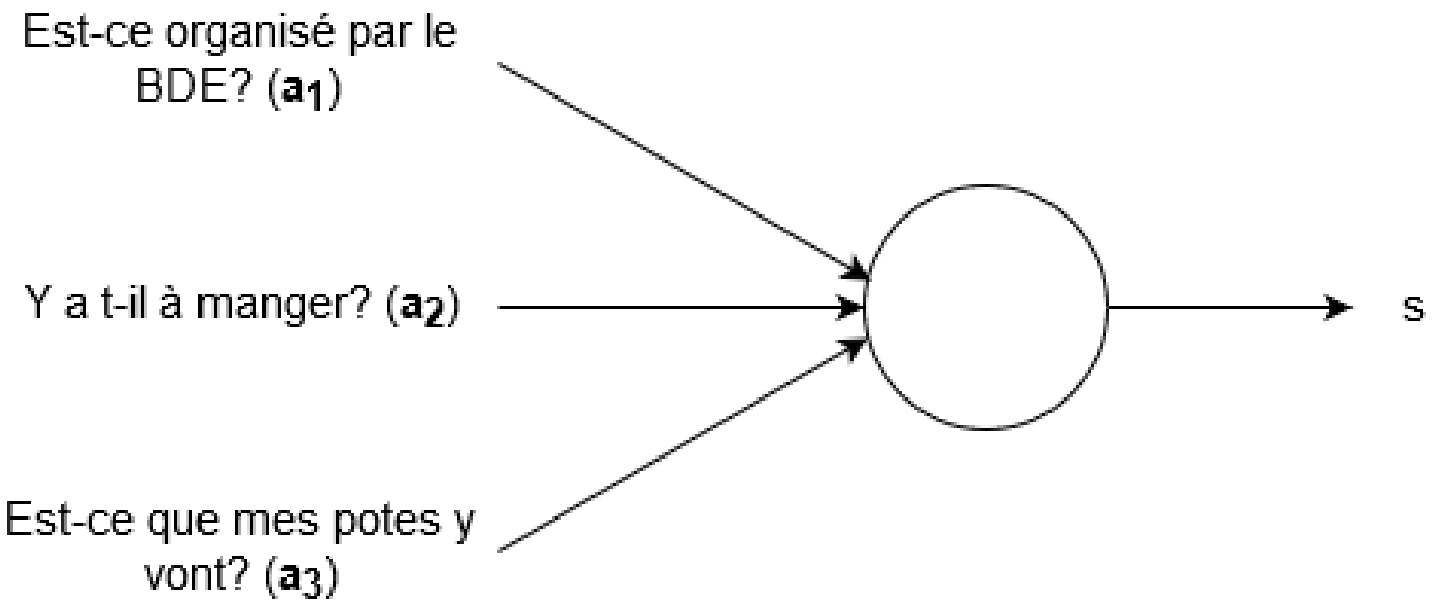
$a_1, ..., a_n, s \in 0, 1$

$$s = \begin{cases} 1 & \text{if } \sum_{i=0}^{n} a_i * w_i + b > 0 \\ 0 & \text{otherwise.} \end{cases}$$

Perceptron - Example

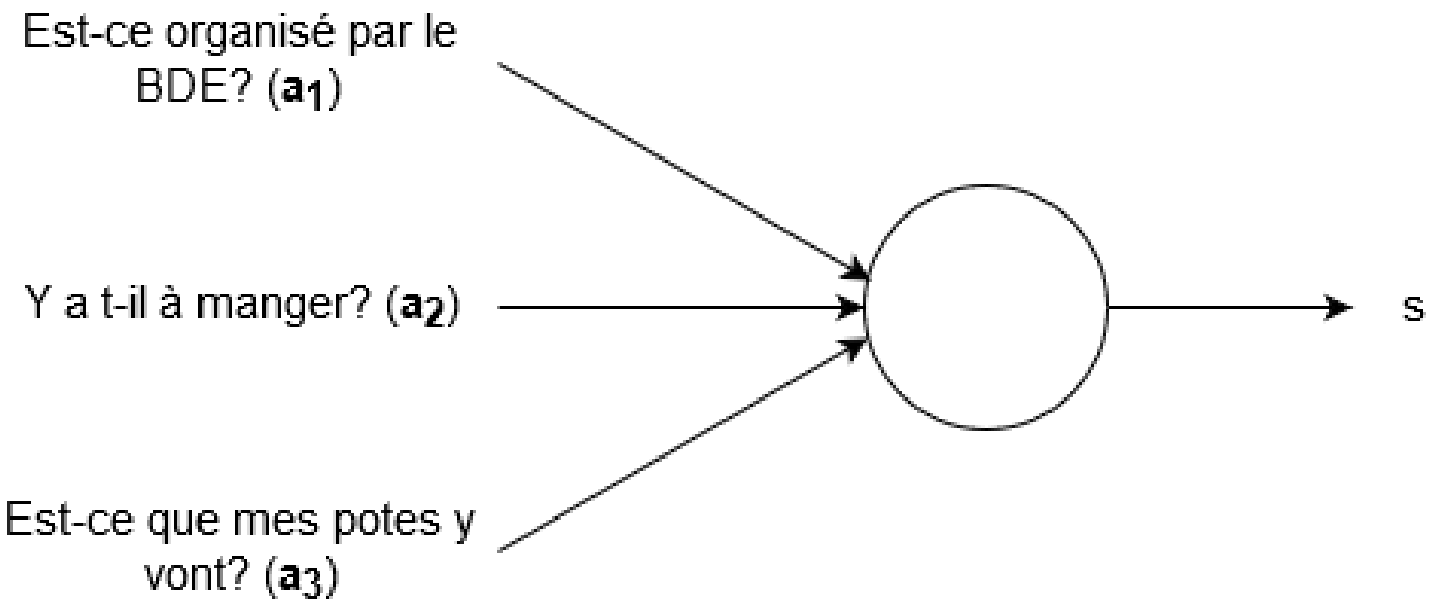


Perceptron - Example



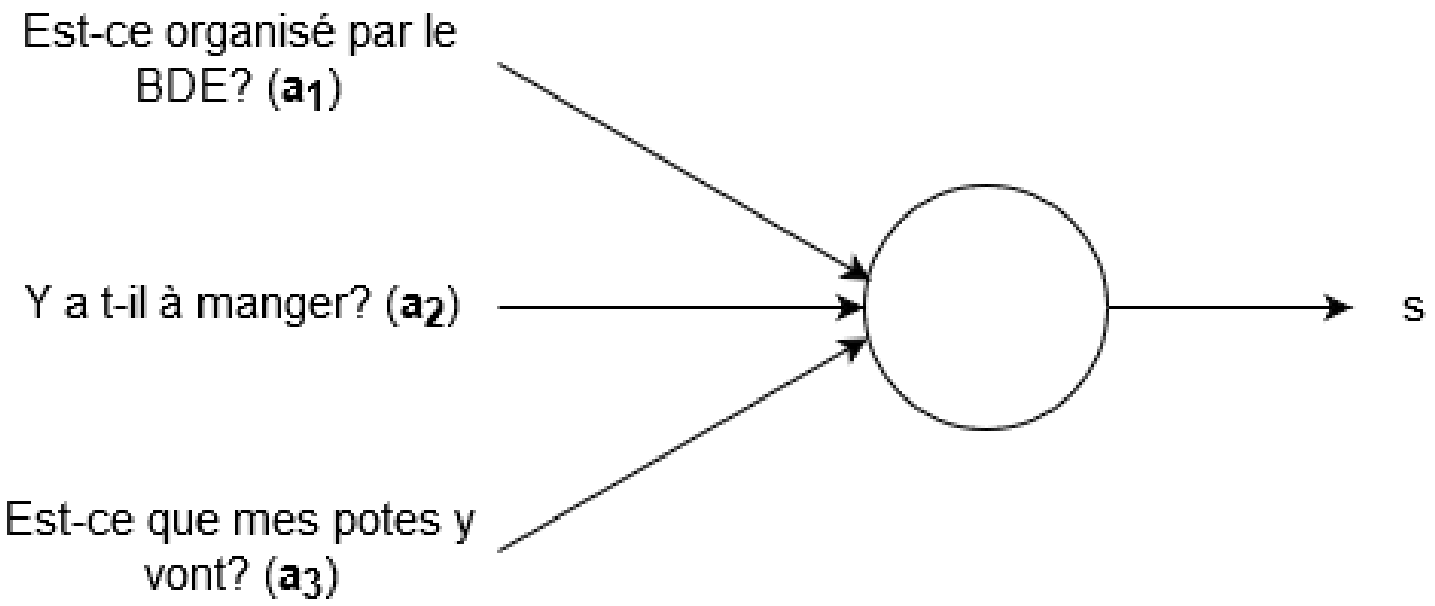
$$\text{If } a_1 + a_2 + a_3 = 3, s = 1$$

Perceptron - Example



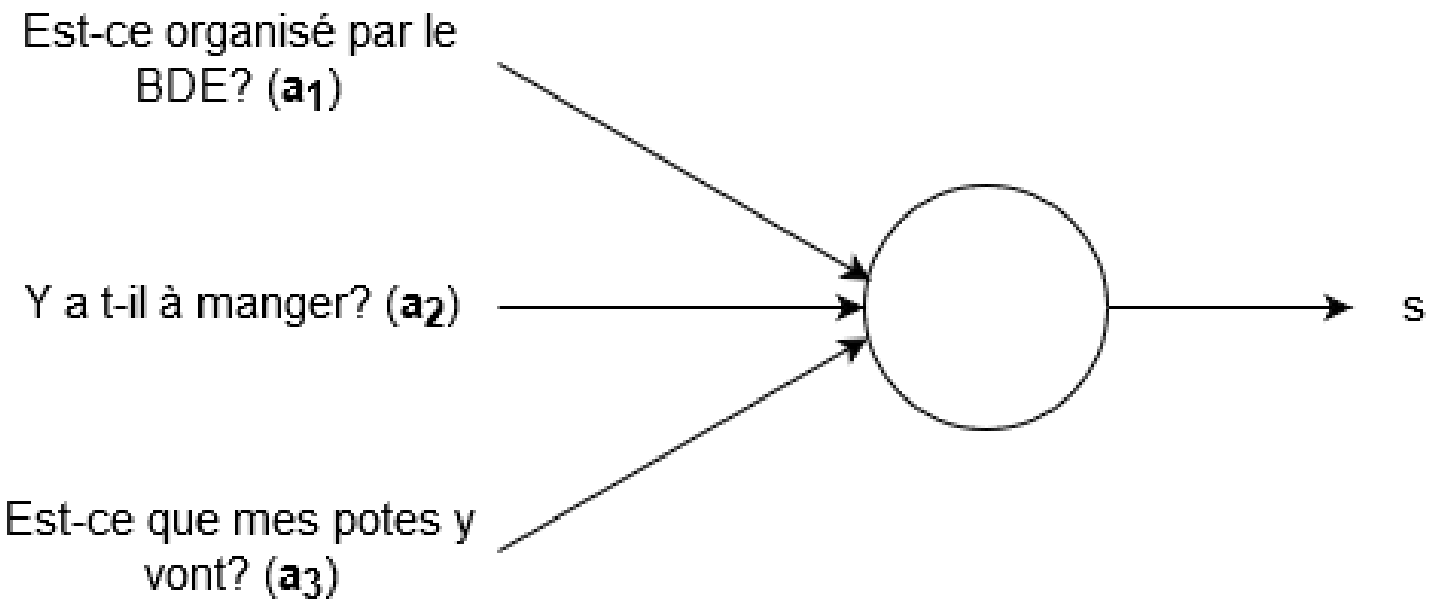
If $a_1 + a_2 + a_3 = 3, s = 1$
We go to the party !

Perceptron - Example



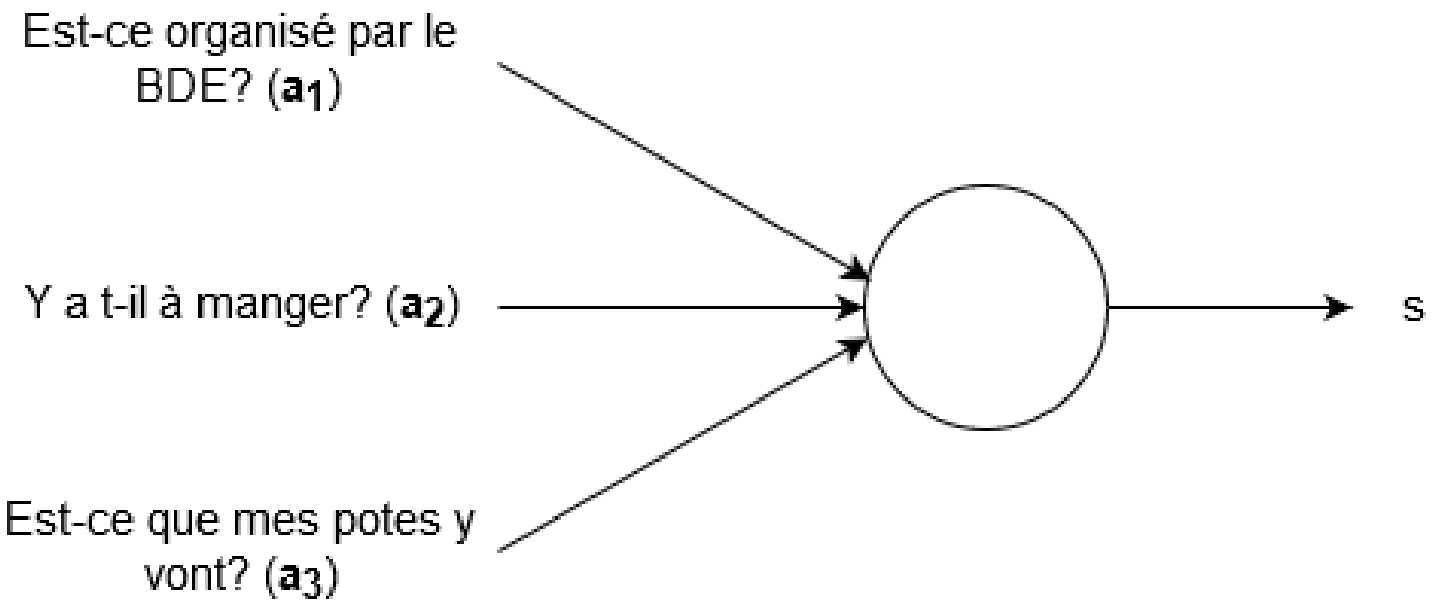
$$\text{If } a_1 + a_2 + a_3 > threshold, s = 1$$

Perceptron - Example



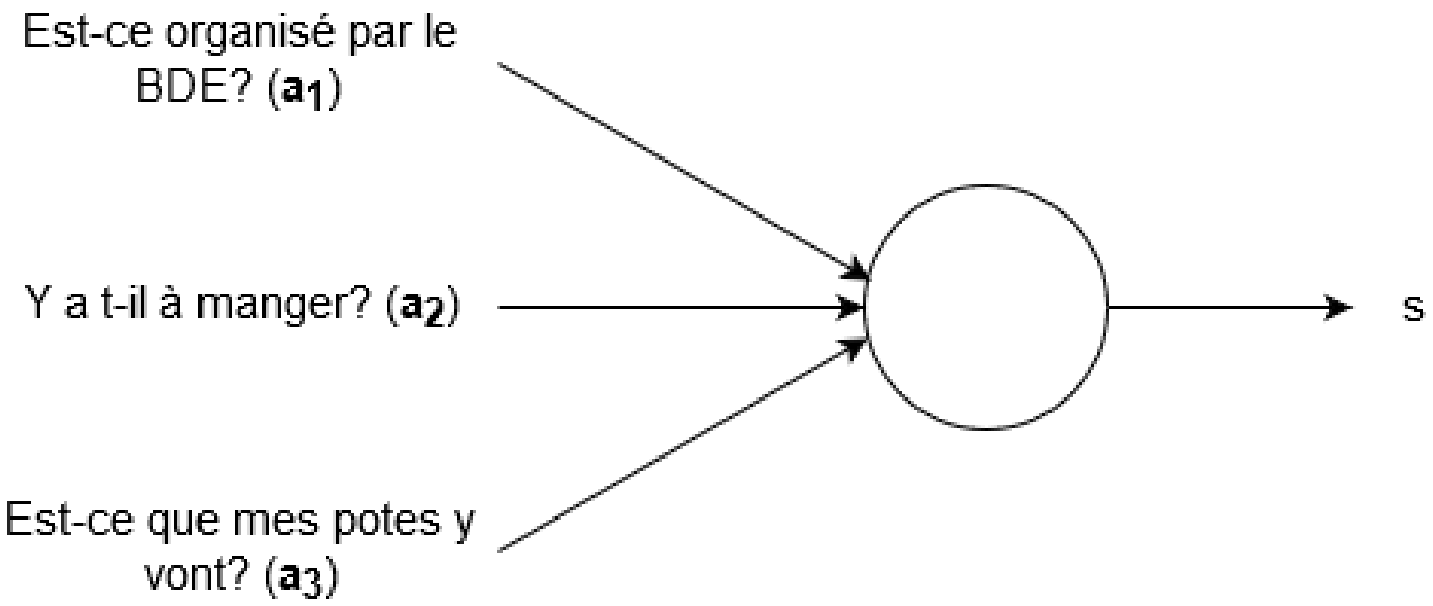
If $a_1 + a_2 + a_3 > threshold, s = 1$
We go to the party !

Perceptron - Example



$$\text{If } a_1 * w_1 + a_2 * w_2 + a_3 * w_3 + b > 0, s = 1$$

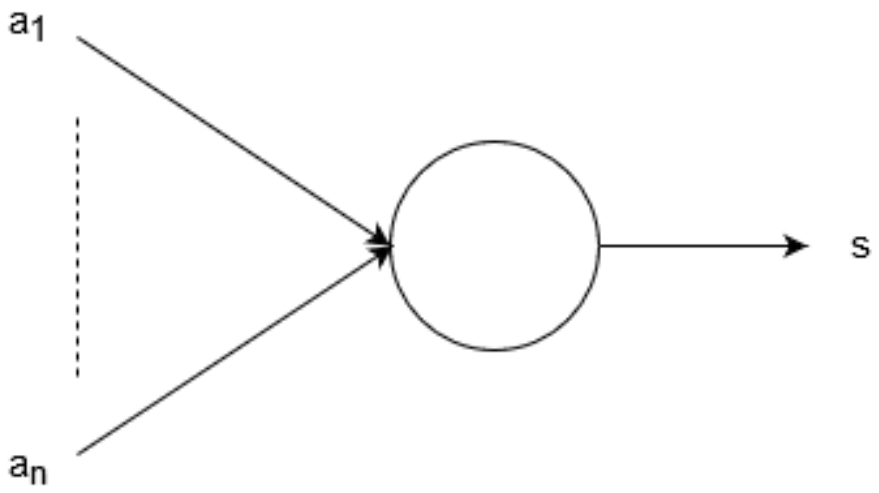
Perceptron - Example



$$\text{If } a_1 * w_1 + a_2 * w_2 + a_3 * w_3 + b > 0, s = 1$$

We go to the party !

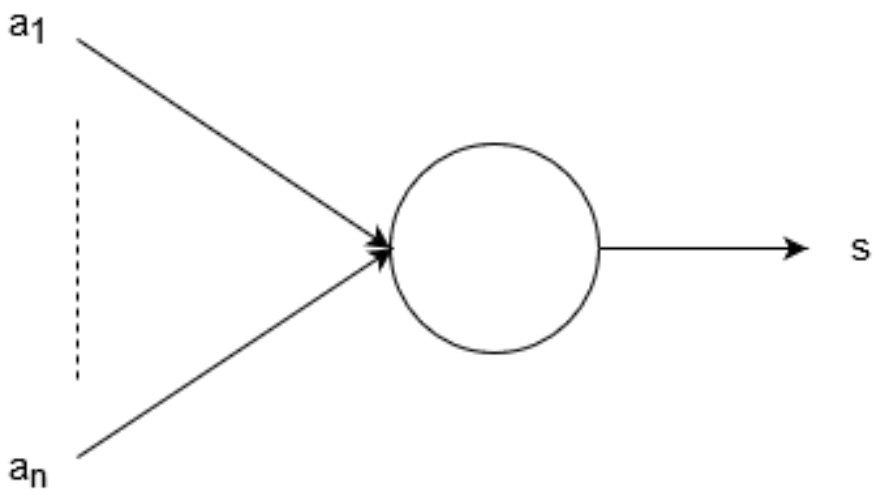
Perceptron



$a_1, ..., a_n, s \in 0, 1$

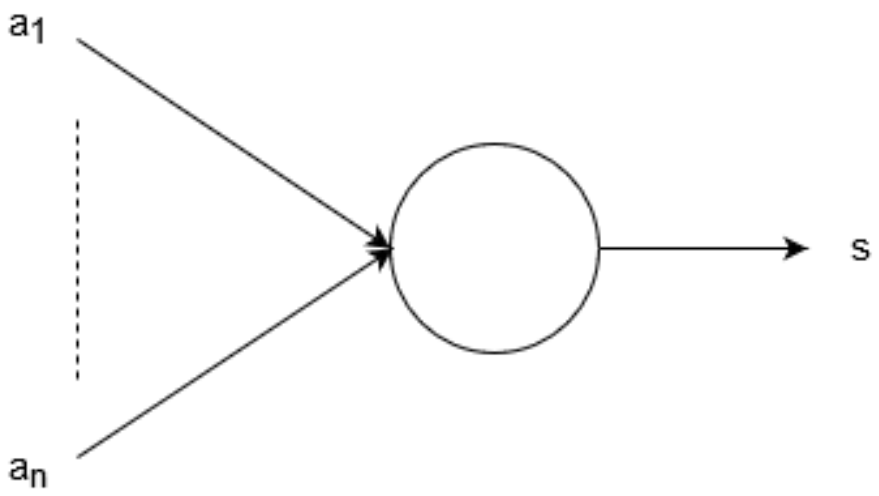
$$s = \begin{cases} 1 & \text{if } \sum_{i=0}^{n} a_i * w_i + b > 0 \\ 0 & \text{otherwise.} \end{cases}$$

Perceptron



→ Capable of reproducing logical gates !
→ Finding the w and b by hand is a pain

Perceptron

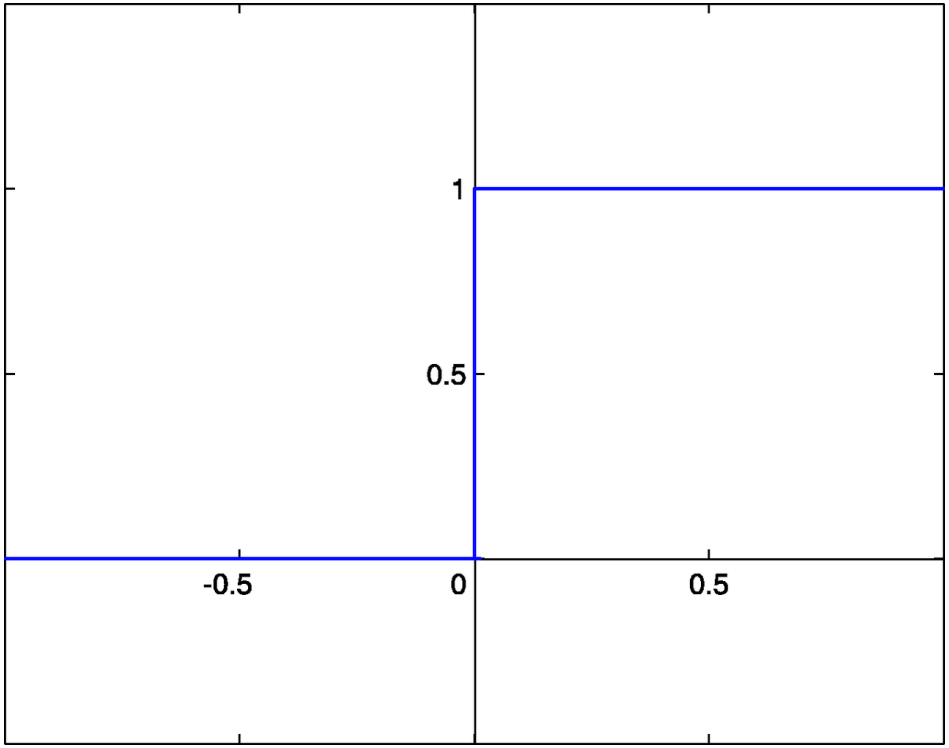


→ Capable of reproducing logical gates !
→ Finding the w and b by hand is a pain

You have to "**learn**" the w and b.

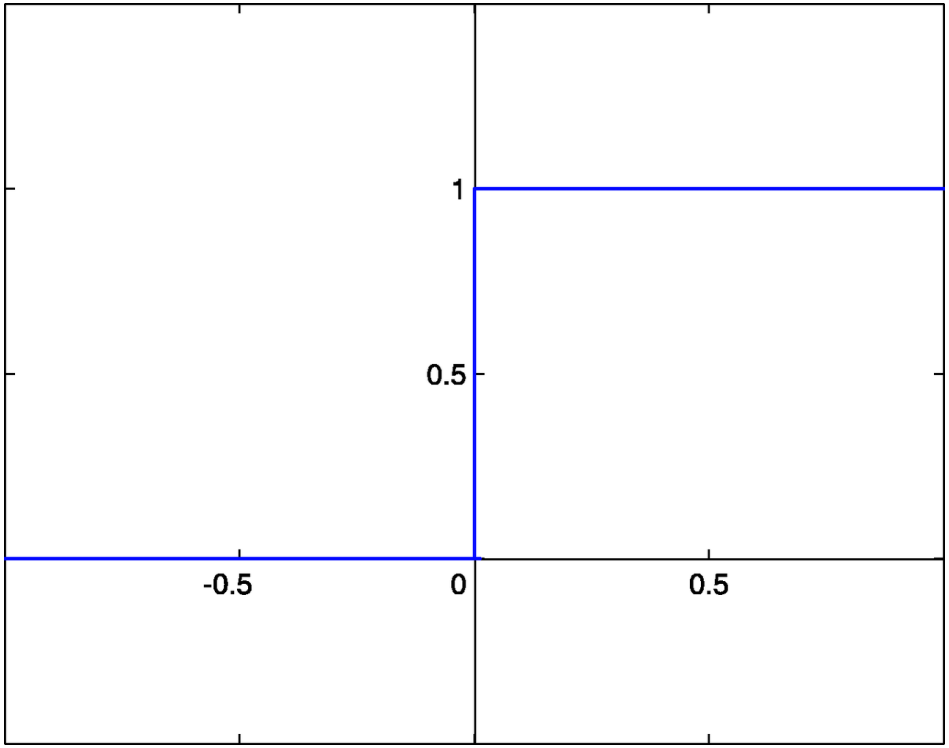
Perceptron

How to learn ?
Small change of w and b $\rightarrow$ small change of the output ?



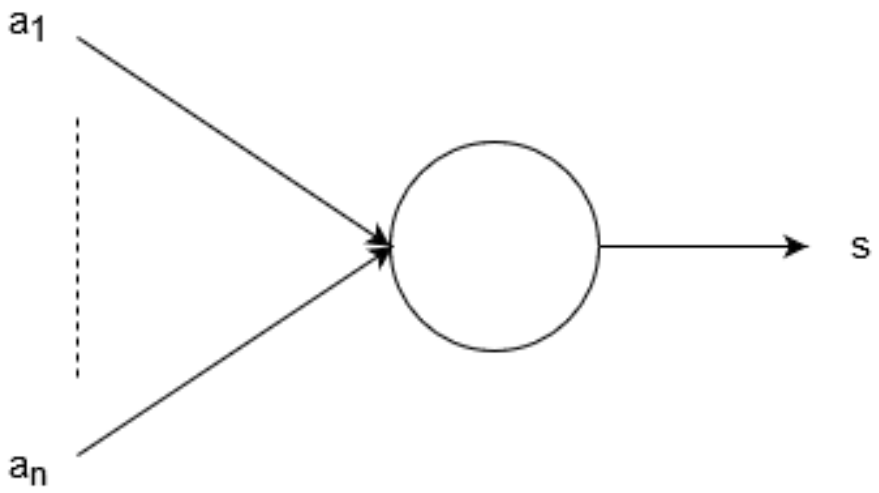
Perceptron

How to learn ?
Small change of w and b $\rightarrow$ small change of the output ?



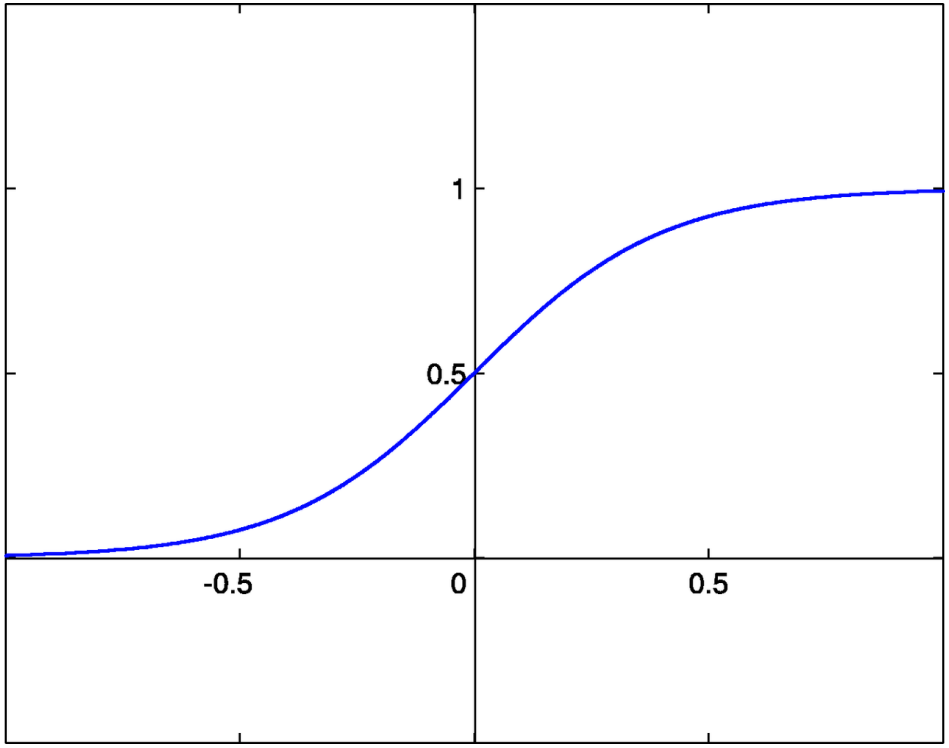
Not possible here

Sigmoid neuron



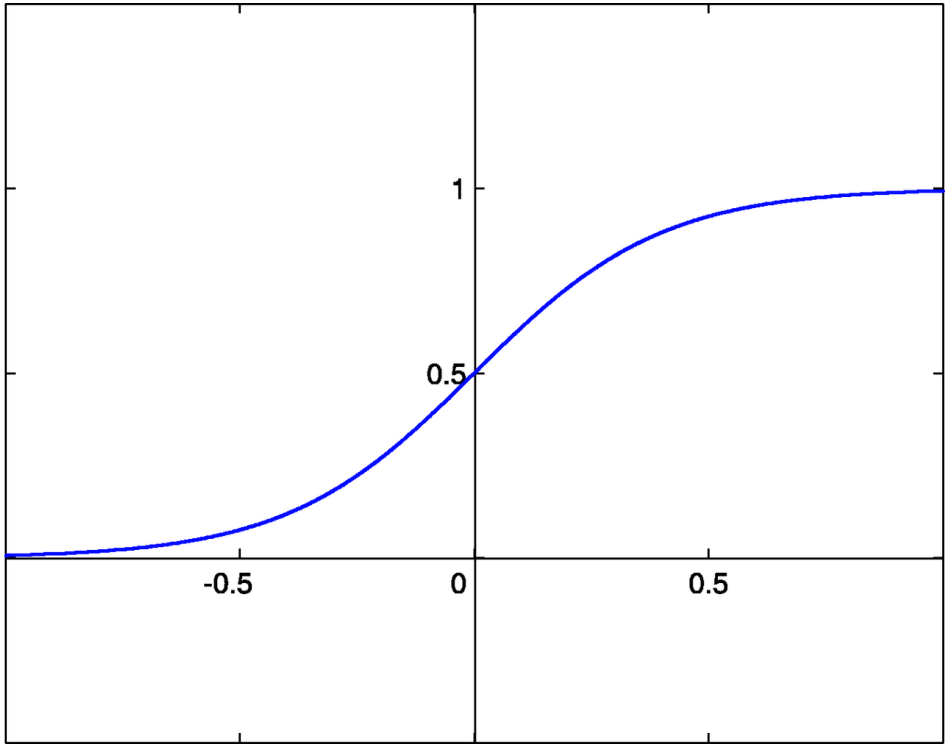
$a_1, ..., a_n \in [0, 1]$
$s = \sigma(\sum_{i=0}^{n} a_i * w_i + b)$ with $\sigma(x) = \frac{1}{1+e^{-x}}$

Sigmoid neuron



Small change of w and b $\rightarrow$ small change of the output ✓

Sigmoid neuron



Small change of w and b $\rightarrow$ small change of the output ✓

Goal how do we train ?

Training

Objective : minimize the error on the predictions !

Training

Objective : minimize the error on the predictions !

$$\left\{\omega, b | E(\omega, b) = \min_{\omega', b'} E(\omega', b')\right\}$$

Training

Problems to solve :

- How to quantify the error ?

Training

Problems to solve :

- How to quantify the error ?
  $\rightarrow$ *Quadratic loss* :

$$L = \frac{1}{n} \sum (desired - predicted)^2$$

Training

Problems to solve :

- How to quantify the error ?
  $\rightarrow$ *Quadratic loss* :
  $$L = \frac{1}{n} \sum (desired - predicted)^2$$
- How to minimize ?

Training

Problems to solve :

- How to quantify the error ?
  $\rightarrow$ *Quadratic loss* :
  $$L = \frac{1}{n} \sum (desired - predicted)^2$$
- How to minimize ?
  $\rightarrow$ *Backpropagation*

Training

Gradient Descent

Training

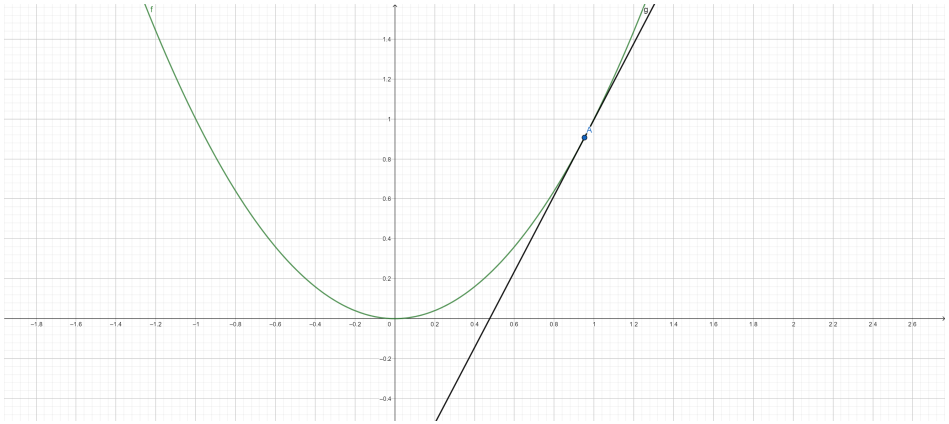
Gradient Descent

Idea : reach the minimum of a function iteratively

Training

Gradient Descent

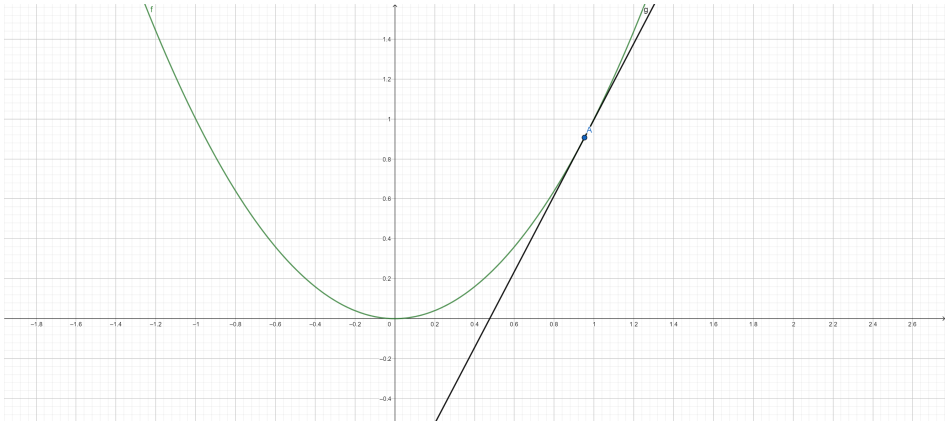
Idea : reach the minimum of a function iteratively

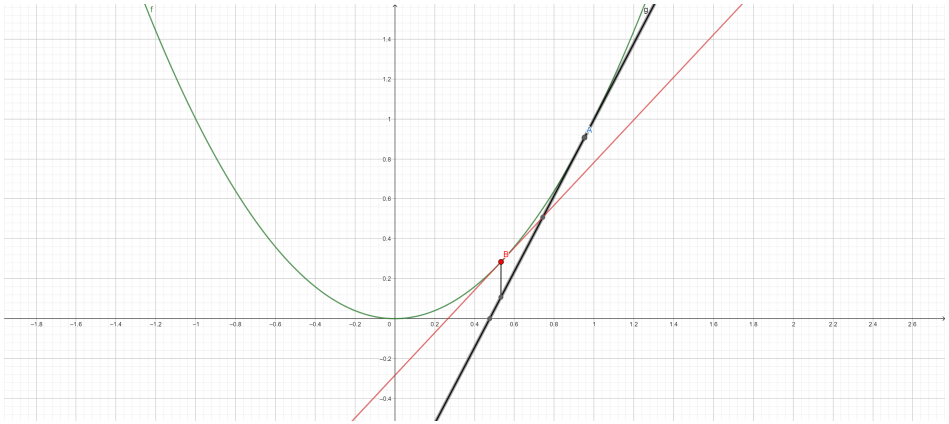


Training

Gradient Descent

Idea : reach the minimum of a function iteratively





Training

For each neuron :

$$\omega' = \omega - \eta \frac{\partial L}{\partial \omega} \tag{1}$$

$$b' = b - \eta \frac{\partial L}{\partial b} \tag{2}$$

Training

For each neuron :

$$\omega' = \omega - \eta \frac{\partial L}{\partial \omega} \tag{1}$$

$$b' = b - \eta \frac{\partial L}{\partial b} \tag{2}$$

Objective : Compute ∇L

Training

For each neuron :

$$\omega' = \omega - \eta \frac{\partial L}{\partial \omega} \tag{1}$$

$$b' = b - \eta \frac{\partial L}{\partial b} \tag{2}$$

Objective : Compute ∇L
$\rightarrow$ one layer ✓

Training

For each neuron :

$$\omega' = \omega - \eta \frac{\partial L}{\partial \omega} \tag{1}$$

$$b' = b - \eta \frac{\partial L}{\partial b} \tag{2}$$

Objective : Compute ∇L
$\rightarrow$ one layer ✓
$\rightarrow$ multiple layers :

Training

For each neuron :

$$\omega' = \omega - \eta \frac{\partial L}{\partial \omega} \tag{1}$$

$$b' = b - \eta \frac{\partial L}{\partial b} \tag{2}$$

Objective : Compute ∇L
$\rightarrow$ one layer ✓
$\rightarrow$ multiple layers : Propagation of the gradient upstream of the network with the chain rule : *Backpropagation* $\rightarrow$ **cs231**

Training

- **1st approach**
  For each entry :
  → Compute the error (loss)
  → Compute the gradient
  → Update the parameters

Training

- **1st approach**
  For each entry :
  → Compute the error (loss)
  → Compute the gradient
  → Update the parameters
- **2nd approach**
  For each *batch* :
  → Compute the mean error
  → Compute the gradient
  → Update the parameters

Training

Summary :
→ Select a *batch*
→ For each entry, compute the output : *Forward propagation*
→ Compute the mean error (loss)
→ Compute the gradient and update the parameters : *Backpropagation*